Лабораторная работа
Параллельный алгоритм быстрой сортировки (метод регулярного набора образцов) с использованием OpenMP

Студент группы ИА332 Рубцов Роман Кириллович

Новосибирск, 2026

1 Цель работы

Реализация параллельного алгоритма быстрой сортировки методом регулярного набора образцов (Parallel Sorting by Regular Sampling — PSRS) с помощью технологии OpenMP. Проверка корректности сортировки, экспериментальное исследование масштабируемости алгоритма на массивах размерности 10 000, 20 000, 30 000 и 40 000 элементов при различном числе потоков, а также теоретическое обоснование полученного ускорения.

2 Конфигурация

- **Процессор:** Intel Core i5-10400F (6 ядер, 12 потоков).
- **Среда разработки:** GCC (флаги `-O3 -fopenmp`).
- **Размеры массивов:** $N \in \{10\,000,\ 20\,000,\ 30\,000,\ 40\,000\}$.
- **Тип данных:** `double`.

3 Общее описание параллельного алгоритма

В работе реализован алгоритм параллельной сортировки методом регулярного набора образцов (PSRS). Принцип работы алгоритма для p потоков и массива из N элементов состоит из четырех последовательных фаз, разделенных барьерной синхронизацией:

3.1 1. Локальная сортировка и выборка образцов

Исходный массив условно разбивается на p непрерывных блоков размера $\approx N/p$. Каждый поток с идентификатором $\mathrm{tid} \in [0, p-1]$ независимо сортирует свой блок с помощью быстрой

сортировки (`std::sort`). После этого каждый поток равномерно с шагом $\lfloor N/p^2 \rfloor$ отбирает из своего отсортированного блока p элементов (регулярных образцов) и записывает их в общий массив образцов размера p^2.

3.2 2. Выбор ведущих элементов-разделителей (Splitters)

После прохождения барьера синхронизации (`#pragma omp barrier`) один поток внутри директивы `#pragma omp single` сортирует массив из p^2 собранных образцов и выбирает из него $p-1$ равноотстоящих элементов с индексами $p, 2p, \ldots, (p-1)p$. Эти элементы выступают глобальными разделителями (сплиттерами), которые разбивают весь диапазон значений на p непересекающихся интервалов (классов).

3.3 3. Разбиение локальных блоков на классы

Каждый поток с помощью бинарного поиска (`std::upper_bound`) находит позиции всех $p-1$ разделителей внутри своего отсортированного локального подмассива. Таким образом, локальный блок каждого потока оказывается разделенным на p подмножеств.

3.4 4. Глобальный обмен и финальная сортировка

В однопоточной секции (`#pragma omp single`) подсчитываются суммарные размеры каждого из p классов и начальные индексы для записи в результирующий массив. Затем каждый поток tid параллельно копирует из всех p локальных блоков части, относящиеся к классу tid, в свою область результирующего массива и выполняет финальную быструю сортировку собранного класса.

4 Результаты проверки правильности сортировки

Для верификации корректности работы параллельного алгоритма после каждого тестового прогона выполнялась автоматическая проверка отсортированного массива функцией `check_sorted`, проверяющей выполнение инварианта неубывания $A[i-1] \leq A[i]$ для всех $i \in [1, N-1]$. На всех исследованных размерах массива ($10\,000 \ldots 40\,000$) и при любом количестве потоков ($1 \ldots 32$) нарушений порядка следования элементов не обнаружено.

5 Результаты исследования масштабируемости

Измерения времени выполнения проводились путем усреднения по 50 запускам для минимизации влияния системных флуктуаций.

Таблица 1: Время выполнения и ускорение $S_p = T_1/T_p$ для 1, 2, 4 и 8 потоков

Количество элементов	**1 поток**	**2 потока**		**4 потока**		**8 потоков**	
	время (мс)	время (мс)	ускорение	время (мс)	ускорение	время (мс)	ускорени
10 000	0.441	0.356	1.24x	0.172	2.56x	0.112	3.92x
20 000	0.939	0.752	1.25x	0.355	2.64x	0.237	3.95x
30 000	1.487	1.159	1.28x	0.558	2.67x	0.353	4.22x
40 000	2.003	1.706	1.17x	0.872	2.30x	0.549	3.65x

Таблица 2: Полная таблица времени выполнения алгоритма (в миллисекундах)

Потоки	**Size 10 000**	**Size 20 000**	**Size 30 000**	**Size 40 000**
1	0.44056	0.93894	1.48732	2.00264
2	0.35556	0.75218	1.15890	1.70568
4	0.17198	0.35524	0.55802	0.87176
6	0.14120	0.30030	0.45186	0.69170
8	0.11248	0.23748	0.35284	0.54858
16	0.20442	0.36218	0.44710	0.65374
32	0.35084	0.47328	0.59388	0.79978

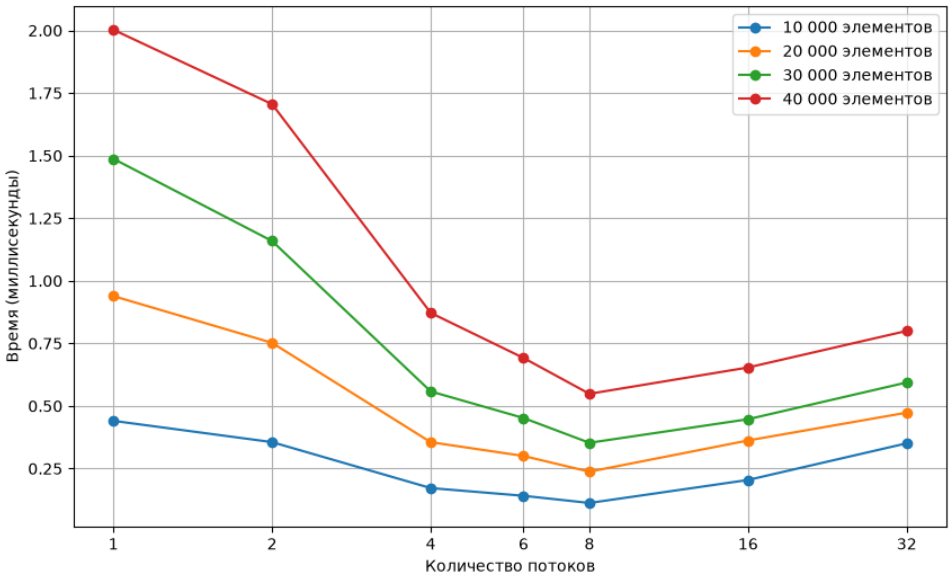


Рис. 1: График зависимости времени сортировки от числа потоков

6 Теоретическое исследование (обоснование) ускорения

6.1 1. Оценка вычислительной сложности

Временная сложность последовательной быстрой сортировки массива из N элементов составляет:

$$T_1 = O(N \log_2 N)$$

В параллельном алгоритме PSRS при использовании p потоков время выполнения складывается из следующих составляющих:

1. **Локальная быстрая сортировка блоков размера N/p:**

$$T_{\text{local}} = O\left(\frac{N}{p} \log_2 \left(\frac{N}{p}\right)\right)$$

2. **Сортировка p^2 образцов в однопоточном режиме:**

$$T_{\text{samples}} = O(p^2 \log_2(p^2)) = O(p^2 \log_2 p)$$

3. **Бинарный поиск $p-1$ разделителей в подмассиве размера N/p:**

$$T_{\text{search}} = O\left(p \log_2 \left(\frac{N}{p}\right)\right)$$

4. **Копирование и финальная сортировка собранного класса:** благодаря регулярной выборке образцов размер каждого класса гарантированно не превышает $\frac{2N}{p}$ элементов, поэтому этап занимает $O\left(\frac{N}{p} \log_2 \left(\frac{N}{p}\right)\right)$.

Таким образом, суммарная теоретическая сложность параллельного алгоритма составляет:

$$T_p = O\left(\frac{N}{p} \log_2 \left(\frac{N}{p}\right) + p^2 \log_2 p\right)$$

Теоретическое ускорение определяется отношением:

$$S_p = \frac{T_1}{T_p} \approx \frac{N \log_2 N}{\frac{c_1 N}{p} \log_2(N/p) + c_2 \frac{N}{p} + c_3 p^2 \log_2 p + T_{\text{sync}}(p)}$$

где $c_1 \approx 2$ обусловлено двукратным применением локальной сортировки (на 1-м и 4-м этапах алгоритма), член $c_2 \frac{N}{p}$ отражает затраты на копирование памяти в буфер результата, а $T_{\text{sync}}(p)$ — накладные расходы на четыре барьерные синхронизации OpenMP.

6.2 2. Сопоставление теории с экспериментальными данными

- **Причины сублинейного ускорения на 2 потоках ($S_2 \approx 1.17$–1.28):** в отличие от однопоточного вызова `std::sort`, параллельный алгоритм PSRS выполняет две фазы

сортировки (начальную и финальную после слияния классов), а также полное копирование массива в дополнительный буфер `result`. Из-за удвоения константы вычислительной сложности на 2 потоках выигрыш от распараллеливания частично компенсируется дополнительными операциями перемещения данных.

- **Рост эффективности на 4–8 потоках ($S_8 \approx 3.65$–4.22):** по мере роста числа потоков до 8 объем локальных подмассивов N/p уменьшается в 8 раз, они полностью помещаются в быстрый кэш L1/L2 каждого физического ядра, а логарифмический множитель $\log_2(N/p)$ сокращает число сравнений. Максимум производительности на всех размерах достигается именно на 8 потоках (например, для $N = 30\,000$ время снижается с 1.487 мс до 0.353 мс, давая ускорение в 4.22 раза).

- **Деградация на 16 и 32 потоках:** при превышении числа аппаратных потоков процессора ($p > 12$) начинает доминировать последовательное слагаемое сортировки образцов $O(p^2 \log_2 p)$ (при $p = 32$ массив образцов достигает $p^2 = 1024$ элементов, что сопоставимо с размером самих блоков N/p), а также резко возрастают задержки $T_{\text{sync}}(p)$ на четырех барьерах синхронизации и переключениях контекста ОС.

7 Вывод

Реализован и исследован параллельный алгоритм быстрой сортировки методом регулярного набора образцов (PSRS) с использованием OpenMP. Экспериментально подтверждена корректность сортировки и хорошая масштабируемость алгоритма вплоть до 8 потоков, где достигнуто максимальное ускорение до 4.22 раза. Показано, что при дальнейшем увеличении числа потоков (16 и 32) на массивах малой размерности ($N \leq 40\,000$) производительность снижается вследствие квадратичного роста объема выборки образцов $O(p^2 \log_2 p)$ и накладных расходов на барьерную синхронизацию потоков.